BoxOffice Brain

It runs real ClickHouse SQL, and it will not invent a box-office number.

doom2quake · Agentic Cinema 2026 · ClickHouse + Google Cloud track

The problem

A studio analyst asks a plain-English catalogue question and gets a confident answer.

The one thing they cannot do is walk into a greenlight meeting with a number they cannot defend.

Generic text-to-analytics agents invent box-office figures with total confidence. You cannot tell a grounded "War fell 0.36 rating points" from a hallucinated one.

The pain is not "writing SQL is hard". It is "an analyst cannot ship a figure a hallucination might have produced".

What it does

You ask in plain English. BoxOffice Brain runs a four-step pipeline:

1. **Plans** the analysis with Gemini and describes the ClickHouse tables it is allowed to see.
2. **Writes** a single read-only SQL statement.
3. **Runs** it through the official ClickHouse MCP server's `run_query` tool.
4. **Answers** using only the rows that came back, with a grounding trace, or abstains and names the data it would need.

Every number on screen traces to a row the database returned. The refusal is the product, not a caveat.

Architecture

```
   Studio question (plain English)
             |
      Gemini / ADK supervisor  ---- plan --->  read-only ClickHouse SQL
             |                                          |
             |                        [ READ_ONLY_SQL · QUERY_SCOPE allowlist
             |                          ACTION_LIMITER · server-side bounds ]
             |                                          |
             |                          official ClickHouse MCP server (run_query)
             |                                          |
             |                                  ClickHouse warehouse
             |                                          |
      rows + grounding trace  <-------------------------+
             |
    grounded answer   OR   abstain: "insufficient comparable data"
```

The guardrails wrap the SQL. The MCP `run_query` call sits on the critical path between the plan and the answer.

The ClickHouse MCP `run_query` call is load-bearing

```
$ boxoffice mcp-check
launching official ClickHouse MCP server: uvx --python 3.11 mcp-clickhouse
endpoint: https://sql-clickhouse.clickhouse.com:8443/ (user=demo)
tools advertised: ['list_databases', 'list_tables', 'run_query']
tool call: run_query   latency: 1030.7 ms   rows: 5
```

Those tool names are the server's, not ours. `mcp-clickhouse` is ClickHouse's own package, not a wrapper with an MCP-shaped name.

No credentials: the public ClickHouse SQL playground, IMDb catalogue, 388,269 films. Captured in docs/live-proof.txt.

Three transports, one statement, measured not claimed

`BOX_TRANSPORT`	runs the SQL	needs
`mcp`	official ClickHouse MCP server, then ClickHouse	`uvx`, network
`http`	ClickHouse HTTP, same SQL, no MCP hop	network
`offline`	SQLite over a committed extract of the same tables	nothing

```
$ python scripts/verify_parity.py
offline vs http: IDENTICAL   offline vs mcp: IDENTICAL   http vs mcp: IDENTICAL
```

18 rows, every numeric cell equal to within **1e-9**. Offline executes the SQL: `WHERE 1=0` returns zero rows.

It refuses to lie

It abstains, and it separates no data from the query failed:

```
Ask about 2021 (outside the snapshot)  -> query runs, no rows -> ABSTAIN, names the gap
Ask about APAC revenue (no template)   -> ABSTAIN before touching the warehouse
```

And the guardrails refuse real attacks, on real tool return values:

```
SELECT * FROM url('http://169.254.169.254/...')  -> rejected: table function not allowed: url()
SELECT count(*) FROM system.users                -> rejected: table not in the allowed data scope
DROP TABLE imdb.movies                            -> rejected: only SELECT/WITH allowed
```

A read-only SELECT that reaches the cloud metadata endpoint is stopped by QUERY_SCOPE. Every decision is persisted to the run's audit trail.

Google Cloud, at runtime

Gemini (`gemini-2.5-flash`) on **Vertex AI**, orchestrated by a **Google ADK** supervisor over three skills assembled from our reusable `agent-core` :

- **Planner** reads the permitted schema and states the metric and comparison.
- **Analyst** writes read-only SQL and calls the MCP `run_query` tool.
- **Answer Writer** grounds the answer in the returned rows, or abstains.

Gemini writes the SQL and calls the MCP tool itself. A captured end-to-end Vertex run, including the `agent-core` ActionLimiter capping further queries, is in `docs/gemini-run.txt` .

Planning is the top differentiator in ClickHouse's own agent benchmark. Gemini is used as a planner, not a chatbot.

Proven, and honestly bounded

35 keyless tests pass · **4** live tests against ClickHouse's own MCP server pass.

Data-scope allowlist against table-function and cross-database payloads · offline engine evaluates predicates · a failed query is never read as no data · the signal is scoped to the named cohort · MCP **fails closed** · the UI transcript came from a real run.

Honest limits, all written down in HONESTY.md: the endpoint is the public playground (anyone reproduces it without a key), the data is IMDb catalogue, the offline snapshot covers 1998 to 1999, and there is no holdout evaluation yet.

Impact and what's next

Impact. A studio or distributor analyst gets a defensible, sourced answer to a catalogue question in seconds, without writing SQL and without risking a hallucinated figure. Because it is judged on what is demonstrated, the abstain and the grounding trace are the impact argument.

Next. A private ClickHouse Cloud service in place of the public playground; a holdout evaluation harness over a question distribution; richer query planning; cohort-grounded audience signals.

It runs real ClickHouse SQL, and it will not invent a box-office number.

github.com/doom2quake/boxoffice · Gemini + ADK over the official ClickHouse MCP server